משימה 2 - הסוכן שלא קיים

נושאים: מבנים, מערכי מבנים, מצביעים, `const`, העתקה, `sizeof` ובדיקת הרשאות.

הרצת `./operation_b7` מהתיקייה הראשית יוצרת את `mission2/data/mission2_data.c`. אין לערוך את קובץ הראיות שנוצר.

תדריך מודיעיני

באחת התשדורות שפענחתם הורה מורטי הרשע להסתיר את סוכן 67 מהדוחות הרשמיים של הארגון. בזמן שדופנשמירץ מתכונן להפעיל את הבארשבעינייטור נגד באר שבע, מישהו פועל מאחורי הקלעים - ומורטי עושה מאמץ שלא נדע עליו.

בעקבות הממצא, צוות המודיעין מיקד את המעקב במערכת ניהול הסוכנים. במהלך סנכרון פנימי של הארגון יורטה חבילת נתונים שכללה את מאגר הסוכנים, עדכוני הרשאות ויומן גישה למערכות. אלו הראיות שעומדות לרשותכם כעת.

אבל המאגר והיומן אינם מספרים את אותו הסיפור. חלק מהנתונים שגויים, וההרשאות המופיעות ברשומות אינן מסבירות את כל הפעולות שבוצעו בפועל. לפני שנוכל לקבוע מי נכנס לאן - ובעיקר כיצד - צריך לשחזר את מצב המאגר ולבדוק מחדש את אירועי הגישה.

בתוך התעבורה שיורטה נמצאה גם ההתכתבות הבאה:

> **ד"ר דופנשמירץ:** פלנקטון! מי זה סוכן 67?!
>
> **פלנקטון:** איזה סוכן 67?
>
> **ד"ר דופנשמירץ:** הסוכן שמופיע ביומן הגישה שלנו!
>
> **פלנקטון:** רגע, אני בודק... יש פה עשרה סוכנים. אולי הוא נמצא ב־`agents[67]`?
>
> **מורטי הרשע:** אל תעשה את זה.
>
> **פלנקטון:** `Segmentation fault`.
>
> **ד"ר דופנשמירץ:** מה זה אומר?!
>
> **מורטי הרשע:** שאנחנו צריכים מחלקת משאבי אנוש חדשה.

המשימה שלכם

השב"כ העביר אליכם את חומרי החקירה. האחריות שלכם היא להפריד בין נתונים שהשתבשו לבין עדויות לחדירה אמיתית למערכות הארגון. טעות בניתוח ההרשאות עלולה לגרום לנו להתעלם מגישה מסוכנת - או לרדוף אחרי אירוע שלא היה חריג כלל.

תתחילו בזיהוי הסוכנים הרשומים, תשחזרו את נתוני ההרשאות, תבחנו את יומן הגישה ולבסוף תנסו לחלץ ממנו קוד שיאפשר להמשיך בחקירה. הממצאים שתפיקו יקבעו מה באמת ניתן לומר על פעילותו של סוכן 67.

לצורך כך תממשו ב־C את פונקציות החקירה באמצעות מבנים, מערכים ומצביעים. קובצי הראיות מסופקים לכם; **אין לשנות אותם כדי להתאים את הנתונים לקוד**.

שלב A - איתור סוכן במערך מבנים

המבנה `Agent` כבר מוגדר עבורכם ב־`agents.h`:

```
typedef struct {
    int id;
    char name[32];
    char role[32];
    int clearance;
    int active;
} Agent;
```

ממשו ב־`agents.c`:

```
Agent *find_agent(Agent *agents, size_t count, int id);
```

- חפשו לפי `id`, לא לפי אינדקס. המזהים אינם רציפים.
- עברו על המערך באמצעות מצביע מתקדם או אריתמטיקת מצביעים.
- אם נמצא סוכן, החזירו מצביע **לאיבר המקורי במערך**.
- אם הסוכן אינו קיים, `agents == NULL` או `count == 0`, החזירו `NULL`.
- אין לשנות את המערך, להקצות זיכרון או להחזיר כתובת של עותק מקומי.

לבדיקה, מתוך `/mission2`:

```
make test-a
```

הבדיקה מאמתת גם זהות מצביעים: תשובה עם אותם נתונים אך בכתובת אחרת אינה תשובה נכונה. לאחר הצלחה תוצג תוצאת בדיקת המאגר.

שלב B - שחזור מסד הנתונים

מאגר הסוכנים המקורי הוא ראיה, ולכן אסור לבצע עליו תיקונים ישירות. בחבילת הנתונים נמצאו גם תיקוני הרשאות מאומתים, אך לפני שנוכל להשתמש בהם צריך להכין עותק עבודה בטוח של המאגר.

בקובץ `main.c` כבר הוקצה לשם כך מערך בשם `working`. לפני שתעתיקו אליו את הרשומות, חשבו באמצעות `sizeof` כמה איברי `Agent` הוא מסוגל להכיל. אין להסתמך על מספר קבוע: הקיבולת שאתם מחשבים היא זו שצריכה לעבור לפונקציה `copy_agents` כדי שלא נחרוג מגבולות עותק העבודה.

לאחר יצירת העותק, החילו עליו את תיקוני ההרשאות:

```
typedef struct {
    int agent_id;
    int new_clearance;
```

```
    int new_active;
} AgentPatch;
```

ממשו:

```
int copy_agents(Agent *dest, size_t capacity, const Agent *source, size_t count);

int update_agent(Agent *agent, AgentPatch patch);

int apply_patch(Agent *agents, size_t count, const AgentPatch *patch);
```

`copy_agents`

- העתיקו `count` מבנים שלמים לפי ערך. בכל איטרציה קחו באמצעות `&` מצביע לאיבר המקור ומצביע לאיבר היעד, והעתיקו את המבנה באמצעות `*`. אין ליצור כינוי מצביע למקור.
- הפרמטר `capacity` הוא מספר איברי `Agent` הזמינים ביעד, לא מספר הבתים.
- החזירו `1` בהצלחה ו־`0` עבור מצביע לא תקין או קיבולת קטנה מדי.
- אם `count == 0`, החזירו `1` בלי לגשת למצביעים; גם `NULL` מותר במקרה זה.
- בכישלון אין לכתוב ליעד. לעולם אין לשנות את המקור.

`update_agent`

- הפונקציה מקבלת מצביע לסוכן ותיקון שמועבר **לפי ערך**.
- ודאו שהמצביע תקין, ש־`patch.agent_id` זהה ל־`agent->id` ושערכי התיקון בטווחים המותרים.
- צרו עותק מקומי של הסוכן באמצעות `*`, עדכנו את שדות העותק באמצעות `.`, ואז כתבו את המבנה השלם בחזרה דרך המצביע.
- החזירו `1` בהצלחה ו־`0` בכישלון. בכישלון אין לשנות את הסוכן.

`apply_patch`

- מצאו את הסוכן לפי `patch->agent_id`, והעבירו ל־`update_agent` את הסוכן שנמצא ואת מבנה התיקון לפי ערך.
- הפרמטר `new_clearance` חייב להיות בטווח 0–5 ו־`new_active` חייב להיות `0` או `1`.
- החזירו `1` בהצלחה ו־`0` אם המצביע, הערכים או מזהה הסוכן אינם תקינים.
- בכישלון השאירו את כל המערך ללא שינוי. אין להשתמש בהקצאה דינמית.

לבדיקה:

```
make test-b
```

המאגר המקורי חייב להישאר ללא שינוי. מכאן והלאה יש לבצע את ביקורת הגישה על עותק העבודה לאחר החלת התיקונים.

שלב C - ביקורת יומן הגישה

כל אירוע מתואר באמצעות:

```c
typedef struct {
    int agent_id;
    int resource_id;
    int required_clearance;
    int granted;
    int sequence;
    int encoded_symbol;
} AccessEvent;
```

קבועי המשאבים מוגדרים ב־`agents.h`, ובהם `PORT_8080`, `BGU_CAMERAS`, `CONTROL_SYSTEM`, `AIR_SUPPORT`, `EVIL_MORTY_PRIVATE_COMMS` ו־`SERVER_BACKUP`.

סוכן מורשה רק אם המזהה שלו קיים במסד המתוקן, הוא פעיל ורמת ההרשאה שלו גדולה או שווה ל־`required_clearance`. סוכן חסר אינו מורשה.

ממשו:

```c
int classify_access(const Agent *agents, size_t count, const AccessEvent *event);
```

Constant	Meaning
`ACCESS_OK`	מורשה והגישה אושרה
`ACCESS_DENIED`	לא מורשה והגישה נדחתה כראוי
`ACCESS_UNEXPECTED_DENIAL`	מורשה אך הגישה נדחתה
`ACCESS_UNAUTHORIZED_GRANTED`	לא מורשה אך הגישה אושרה - אירוע חמור
`ACCESS_INVALID`	קלט או אירוע לא תקין

דרישות האימות:

- `agents == NULL` מותר רק כאשר `count == 0`; המצביע `event` חייב להיות תקין.
- `agent_id` חייב להיות חיובי. עבור `agent_id <= 0` החזירו `ACCESS_INVALID`.
- `resource_id` חייב להיות אחד מקבועי המשאבים שהוגדרו מראש.
- `required_clearance` חייב להיות בטווח 0–5, והערך `granted` חייב להיות `0` או `1`.
- `sequence` חייב להיות בטווח 1- עד 7, ו־`encoded_symbol` בטווח 1- עד 25.
- אין לשנות את הסוכנים או את האירוע, ואין להסיר את `const` באמצעות המרה.

השתמשו **בעותק המתוקן** לביקורת הסופית. דחייה נכונה של ניסיון לא מורשה היא `ACCESS_DENIED`, לא פריצה מוצלחת.

לבדיקה:

```
make test-c
```

לאחר שהסיווג פועל כראוי, אפשר לבחון את אירועי הגישה האמיתיים ולזהות אילו מהם דורשים חקירה נוספת.

לאחר השלמת שלבים A–C תוכלו להריץ את החקירה לפי פקודת הקומפילציה שבהמשך. המערכת תציג את ממצאי היומן הקונקרטיים ואת הדיאלוגים שנפתחו בעקבותיהם עוד לפני ששחזור הקוד בשלב D הושלם.

שלב D - שחזור קוד הגישה של סוכן 67

ביומן מסתתרים בדיוק שמונה אירועים השייכים לקוד. אירוע מתאים רק אם:

- `agent_id == 67`.
- סיווגו באמצעות `classify_access` הוא `ACCESS_UNAUTHORIZED_GRANTED`.
- `sequence` הוא ערך ייחודי בטווח 0–7.
- `encoded_symbol` בטווח 0–25.

האירועים מעורבבים בין רשומות אחרות. עבור המיקום `i`, פענחו כך:

```
decoded_char = 'A' + ((encoded_symbol - i + 26) % 26);
```

ממשו:

```
int recover_code(const Agent *agents, size_t agent_count, const AccessEvent
*events,
                                                        size_t
event_count, char *code, size_t capacity);
```

- בהצלחה, כתבו שמונה אותיות גדולות ואחריהן `\0`, והחזירו `1`.
- נדרשת רשומת קוד תקינה אחת בדיוק לכל מיקום 0–7. דחו מיקום חסר או מיקום כפול בין רשומות הקוד התקינות.
- אירוע פגום אינו רשומת קוד תקינה: התעלמו ממנו והמשיכו לסרוק. אם לאחר הסריקה חסר מיקום תקין, השחזור נכשל.
- דחו מצביע לא תקין או קיבולת קטנה מ־9.
- בכישלון החזירו `0`. אם `code != NULL` ו־`capacity > 0`, קבעו `'code[0] = '\0`.
- אין לשנות את מערכי הקלט, לכתוב מחוץ לגבולות או להשתמש בהקצאה דינמית.

לבדיקה:

```
make test-d
```

הפעלת החקירה המלאה

לאחר שכל ארבעת השלבים עוברים, קמפלו והריצו מתוך `/mission2`:

```
make run
```

`support/mission2/lib/libb7investigation.a_/..` היא תשתית מקומפלת שמדפיסה ממצאים ודיאלוגים רק כאשר התנאים המתאימים התגלו. אין לערוך אותה או להגיש אותה. היא אינה ממשת אף אחת מהפונקציות שעליכם לכתוב ב־`agents.c`. קוד הגישה הסופי יוצג בפורמט `B7-M2-XXXXXXXX`.

החקירה עשויה לחשוף מה נעשה באמצעות החשבון של סוכן 67, אבל לא בהכרח מי עומד מאחוריו או מדוע מורטי מגן עליו. המשך החקירה תלוי במה שתצליחו לשחזר מתוך ההתכתבויות הבאות.

דוח החקירה

מלאו את חלק משימה 2 ב־investigation_report.md, לאחר השלמת המשימה והרצת החקירה המלאה.

התייחסו בקצרה לשני נושאים:

- ממצאים: אילו אירועי גישה חריגים גיליתם, ומה אפשר להסיק מהם על סוכן 67? ציינו גם את קוד הגישה.
- הבנה טכנית: הסבירו מדוע find_agent מחזירה מצביע לאיבר המקורי ולא למשתנה מקומי, ומדוע ניתן לחשב את מספר האיברים ב־working באמצעות sizeof מתוך main.c, אך לא מתוך הפרמטר dest של copy_agents.

אין צורך לתאר מחדש את כל שלבי המימוש. תשובות קצרות, מדויקות ומנומקות מספיקות.

הגשה

הגישו את:

- `mission2/agents.c`
- `mission2/main.c`
- `investigation_report.md` עם החלק של משימה 2

אין לשנות את קובצי ה־header והספרייה שבתיקייה `/support/mission2_/..`, את חתימות הפונקציות או את קובצי הנתונים שנוצרו.